1. Problem Title & Link

Title: **LeetCode 1248: Count Number of Nice Subarrays**

Link: https://leetcode.com/problems/count-number-of-nice-subarrays/

2. Problem Statement (Short Summary)

You are given:

- An integer array nums
- An integer k

A **nice subarray** is defined as a subarray that contains **exactly k odd numbers**.

Your task is to **count the number of such nice subarrays**.

Important point:

We are **counting the number of subarrays**, not finding the subarray itself.

3. Examples (Input → Output)

Example 1

Input

nums = [1,1,2,1,1]

k = 3

Output

2

Explanation

Nice subarrays:

[1,1,2,1]

[1,2,1,1]

Each contains **exactly 3 odd numbers**.

Example 2

Input

nums = [2,4,6]

k = 1

Output

0

Explanation

There are **no odd numbers**, so no valid subarrays.

Example 3

Input

nums = [2,2,2,1,2,2,1,2,2,2]

k = 2

Output

16

4. Constraints

1 ≤ nums.length ≤ 50,000

1 ≤ nums[i] ≤ 100,000

1 ≤ k ≤ nums.length

Observations:

- Large input size
- Brute force will be too slow
- Need an **O(n)** solution

5. Core Concept (Pattern / Topic)

Prefix Sum + HashMap

Key idea:

Convert the problem into:

Count subarrays with sum = k

But instead of sum, we count **odd numbers**.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Convert Problem

Replace every number:

odd → 1

even → 0

Example

nums = [1,1,2,1,1]

Converted array

[1,1,0,1,1]

Now problem becomes:

Find **subarrays with sum = k**.

Step 2 — Brute Force

Try every subarray.

```
for i in range(n):
   sum = 0
   for j in range(i,n):
      sum += nums[j]
```

Time Complexity

$O(n^2)$

Too slow.

Step 3 — Prefix Sum Idea

Define:

prefix[i] = number of odd elements up to index i

If

prefix[j] - prefix[i] = k

Then the subarray between them has **k odd numbers**.

Rearranging:

prefix[i] = prefix[j] - k

So we count how many times prefix[j] - k appeared before.

Use a **HashMap** to store prefix frequencies.

7. Visual / Intuition Diagram (ASCII)

Example

nums = [1,1,2,1,1]

k = 3

Convert

[1,1,0,1,1]

Prefix

```
index:   0 1 2 3 4
value:   1 2 2 3 4
```

When prefix becomes **3**, we check:

prefix - k = 3 - 3 = 0

If prefix 0 appeared before → valid subarray.

8. Pseudocode (Language Independent)

```
map = {0:1}

prefix = 0
count = 0
```

```
for num in nums:

    if num is odd:
        prefix += 1

    if (prefix - k) exists in map:
        count += map[prefix-k]

    map[prefix] += 1

return count
```

9. Code Implementation

Python

```
class Solution:
    def numberOfSubarrays(self, nums, k):

        prefix = 0
        count = 0

        freq = {0:1}

        for num in nums:

            if num % 2 == 1:
                prefix += 1

            if prefix - k in freq:
                count += freq[prefix-k]

            freq[prefix] = freq.get(prefix,0) + 1

        return count
```

Java

```
class Solution {
```

```
    public int numberOfSubarrays(int[] nums, int k) {

        HashMap<Integer,Integer> map = new HashMap<>();

        map.put(0,1);

        int prefix = 0;
        int count = 0;

        for(int num : nums){

            if(num % 2 == 1)
                prefix++;

            if(map.containsKey(prefix-k))
                count += map.get(prefix-k);

            map.put(prefix, map.getOrDefault(prefix,0)+1);
        }

        return count;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Single pass through array.

Space Complexity

O(n)

HashMap stores prefix counts.

11. Common Mistakes / Edge Cases

1. Forgetting to initialize map

map[0] = 1

2. Using sum instead of odd count

3. Incorrect prefix update
4. Forgetting modulo check for odd

12. Detailed Dry Run

Example

nums = [1,1,2,1,1]

k = 3

Converted

[1,1,0,1,1]

num	odd?	prefix	prefix-k	map	count
1	1	1	-2	{0:1}	0
1	1	2	-1	{0:1,1:1}	0
2	0	2	-1	{0:1,1:1,2:1}	0
1	1	3	0	map[0]=1	1
1	1	4	1	map[1]=1	2

Final Answer

2

13. Common Use Cases (Real-Life / Interview)

Prefix + HashMap pattern appears in:

- Financial transaction analysis
- Pattern frequency detection
- Streaming analytics
- Network packet analysis

14. Common Traps (Important!)

1. Forgetting prefix initialization
2. Wrong condition for odd numbers
3. Counting incorrect prefix difference
4. Using sliding window instead of prefix

15. Builds To (Related LeetCode Problems)

Prefix + HashMap progression

LC 560 Subarray Sum Equals K

LC 1248 Count Number of Nice Subarrays

LC 974 Subarray Sums Divisible by K

LC 523 Continuous Subarray Sum

LC 525 Contiguous Array

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	$O(n^2)$	Too slow
Prefix + HashMap	$O(n)$	Optimal

17. Why This Solution Works

Prefix sum tracks **total odd numbers seen so far**.

If earlier prefix count was **prefix – k**, then the elements between those indices contain exactly **k odd numbers**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Count subarrays with **at most k odd numbers**
2. Count subarrays with **exactly k even numbers**
3. Find longest subarray with **k odd numbers**
4. Streaming version of the problem
5.